

Module 2

Relational Model

Relational Model

- **Relation**
- **Attribute**
- **Domain**
- **Tuple**
- **Degree**
- **Cardinality**

Relational Model

- The model was first proposed by **Dr. E.F. Codd** of IBM Research in 1970
- Based on the concept of **relation**
- **Relation**
 - Informally, a **relation** looks like a **table** of values. All data and relationship are represented in a table called relation.
 - A relation typically contains a set of **rows**.
- **Tuple**
 - It is row of a relation or table.
 - The data elements in each row represent certain facts that correspond to a real-world entity or relationship
 - Formally, rows are called **tuples**
 - **E.g:** In the student table there are 5 tuples/rows, each row contains 7 values, one for each attribute.
- **Attribute**
 - The **attribute** is **named column** of a relation.
 - Each **column** has a **column header** that gives an indication of the meaning of the data items in that column
 - Formally the column header is called an **attribute name** (or just attribute)

Example of a Relation

Relation Name

STUDENT

Attributes

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	(817)373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	(817)375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53
Rohan Panchal	489-22-1100	(817)376-9821	265 Lark Lane	(817)749-6492	28	3.93
Barbara Benson	533-69-1238	(817)839-8461	7384 Fontana Lane	NULL	19	3.25

Tuples

Relational Model

- **Domain**
 - Defines kind of data represented by an attribute
 - The set of allowed values that an attribute may contain is called the **domain or valueset** of the **attribute**
- **Attribute values** are (normally) required to be **atomic**
- **No two tuples** in a relation are **identical**.
- The special value **null** is a member of every domain.

Relational Model

- **Degree or Arity of a relation:** The number of attribute a relation contains known as degree of that relation
 - A relation with degree 1 -> unary relation
 - A relation with degree 2 -> binary relation
 - A relation with degree 3 -> ternary relation
 - A relation with degree > 3 -> n-ary relation
 - **Example:** Degree of student = 7
- **Cardinality of a relation:** Cardinality of a relation is the number of tuples it contains.
 - It changes as number of tuples are added/deleted.
 - **Example:** Cardinality of student = 5

Relational Model

- Alternate Names

Formal Names	Alternate Name1	Alternate Name2
Relation	Table	File
Tuple	Row	Record
Attribute	Column heading	Field
Domain	Set of possible values of a column	

Relational Database Schema

- A set S of relation schemas that belong to the same database.
- S is the name of the whole **database schema**
- $S = \{R_1, R_2, \dots, R_n\}$ where $R_1, R_2, \dots, R_n$ are the names of the individual **relation schemas** within the database S

Company Database Schema

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

Relational Model Constraints

- 1) **Inherent Model Based or Implicit:** relation can't have duplicate values.
- 2) **Schema Based:** That can be expressed in schema definition through DDL statement.

- a) Domain constraint

- b) Key constraint

- c) Not Null constraint

- d) Unique Constraint

Ex :

- e) Check Constraint

- f) Default constraint

- g) Entity Integrity constraint

- h) Referential integrity constraint

- 3) **Application Based:** Can't be expressed directly in schema definition & has to be enforced through application programs.

Ex: Salary of an employee shouldn't exceed its supervisor.

2) Schema Based Constraints

- a) **Domain Constraint:-** The value of each attribute 'A' must be an atomic value from the domain ***dom(A)***.
- b) **Key Constraint:-** A key has to satisfy three conditions-
- i) In a relation, no two distinct tuples can have identical values for the key attribute.
 - ii) It is a minimal **superkey**.
 - iii) Key attribute can't have NULL values.
- c) **Constraints on Null Values:-** Specifies if the null values are permitted. If not, then that particular attribute is constrained to "**NOT NULL**".

```
CREATE TABLE students (  
  Roll int NOT NULL,  
  LastName varchar(255),  
  FirstName varchar(255),  
  Age int);
```

```
CREATE TABLE students (  
  Roll int UNIQUE,  
  LastName varchar(255),  
  FirstName varchar(255),  
  Age int);
```

2) Schema Based Constraints

d. Unique Constraint: It does not allow duplicate values for the attribute.

```
CREATE TABLE students  
( Roll int NOT NULL UNIQUE);
```

e. Check Constraint: It is a constraint which is applied to the data prior to the insertion in the table.

Ex: Suppose age of a student between 20 to 25 or acct balance should be minimum 500 etc.

f. Default constraint: If any explicit value is not provided to the attribute then it is provided by default value. If no default value is given then it is NULL , if the attribute doesn't have NOT NULL constraint.

```
CREATE TABLE students  
( Roll int NOT NULL UNIQUE  
  age int NOT NULL default 20);
```

2) Schema Based Constraints

g) Entity Integrity Constraint:-

- No primary key value can be NULL.
 - This is because primary key values are used to identify the individual tuples.
 - $t[PK] \neq \text{null}$ for any tuple t in $r(R)$
 - If PK has several attributes, null is not allowed in any of these attributes
- Every primary key attribute is not-NULL and unique
- NULL value has several meanings so it can not able to distinguish between two tuple.

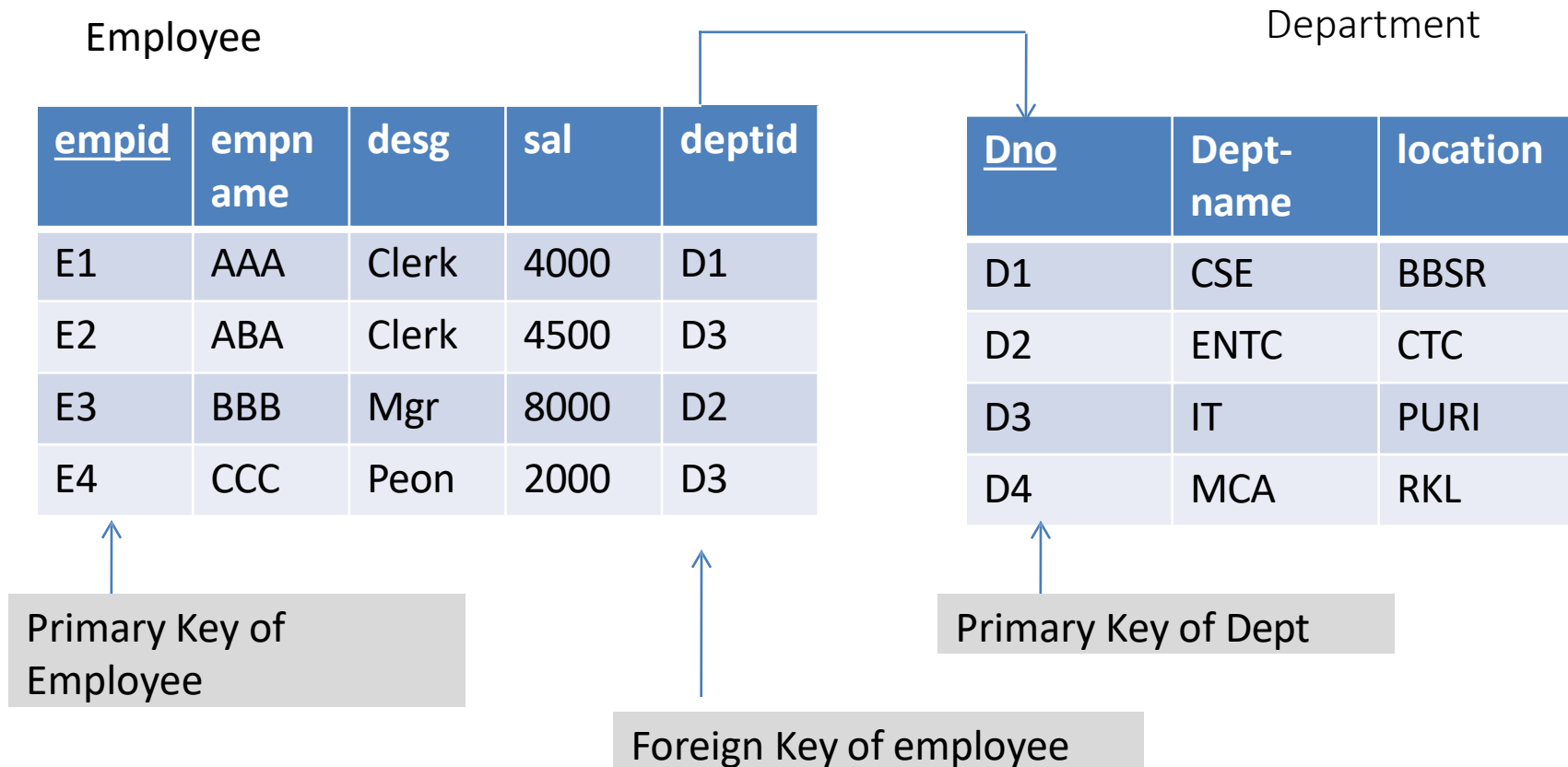
```
CREATE TABLE students (  
    Roll int PRIMARY KEY,  
    LastName varchar(255),  
    FirstName varchar(255),  
    Age int);
```

2) Schema Based Constraints

e) Referential integrity constraints

- Specified between two relations.
- It is based on the concept of “**FOREIGN KEY**”.
- A set of attributes **FK** in relation schema R_1 is a foreign key of R_1 that references relation R_2 if it satisfies the following two conditions :
 - The attribute in FK have the same domain as the primary key attributes PK of R_2 .
 - A value of FK in a tuple t_1 of the current state $r_1(R_1)$ either occurs as a value of PK for some tuple t_2 in the current state $r_2(R_2)$ or is NULL.
- R_1 is called the “referencing relation” and R_2 is the “referenced relation”.

A relation schema may have an attribute that corresponds to the primary key of another relation. The attribute is called a foreign key



Foreign Key are used to link two or more table which have some form of relationship .
Foreign key must values should always match with the corresponding primary key value.

Relational Algebra

- Query: A query is a statement requesting the retrieval of information.
- Query languages: Language in which user **requests information** from the **database**. Allow manipulation and **retrieval of data** from a database.
- There are two types of formal query languages for the relational model.
 - **Procedural Query language**. Describe the operation how to get the desired information.(ex- **Relational Algebra**)
 - **Nonprocedural Query language** : Describe what information the result should contain, rather than how to compute it. (ex- **Relational Calculus**)
 - Tuple Relational Calculus(TRC)
 - Domain Relational Calculus(DRC)

Relational Algebra

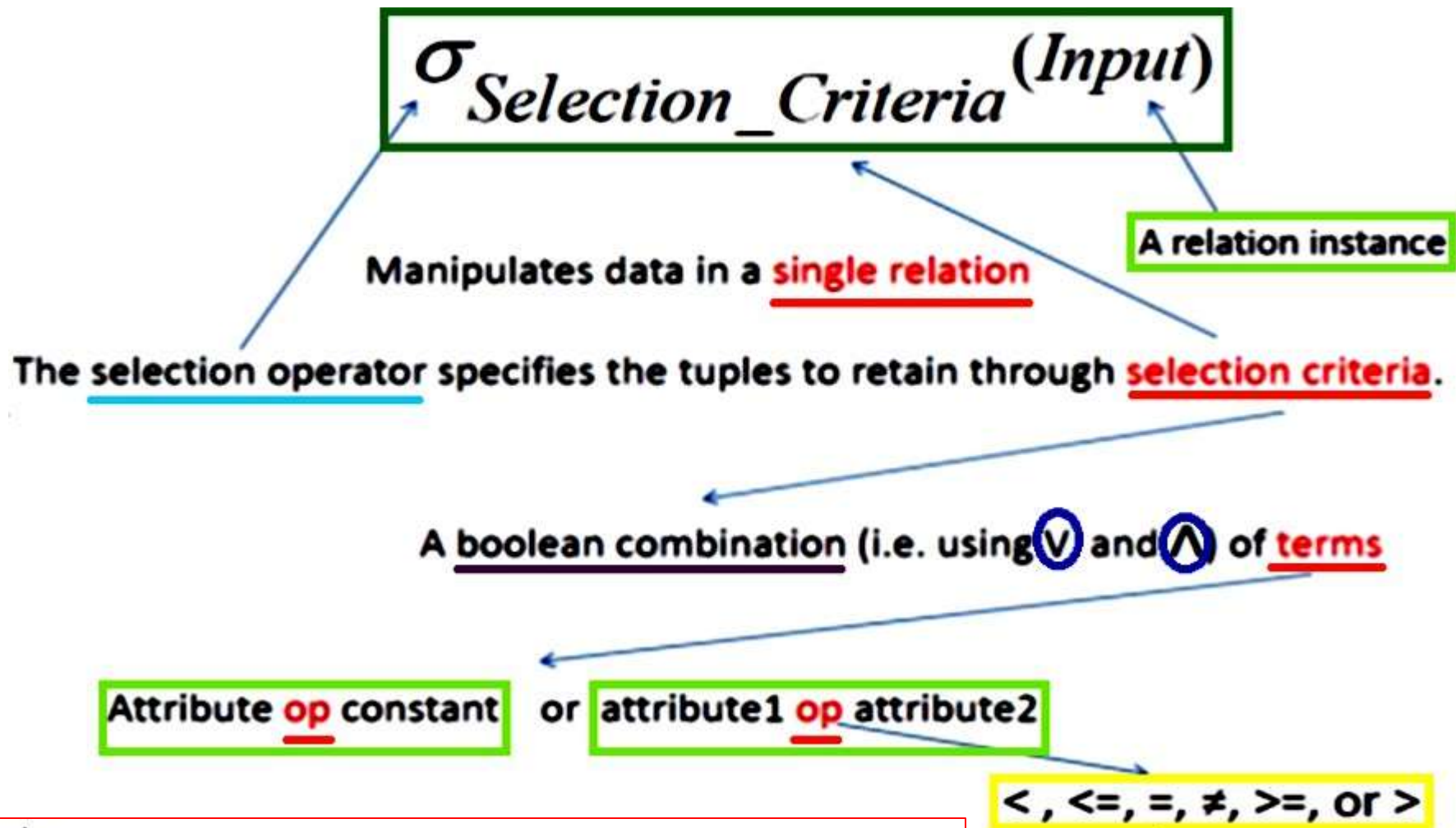
- The **relational algebra** is a **procedural query language**.
- It consists of a **set of operations** that take **one or two relations** as input and produce a **new relation** as their result.
- These operations enable a user to specify **basic retrieval requests (or queries)**.
- It is used to tell the DBMS how to build a new relation from one or more existing relation.
- It forms the basis of Query language.

Relational Algebra

- Six Basic operators:
 - Select
 - Project
 - Cartesian Product or Cross-product
 - Set-difference
 - Union
 - Intersection
- Additional operators
 - Joins (natural, equi-join, theta join, outer-join)
 - Division
 - Renaming
- The relation algebra operators are classified into either
 - **Unary**: Operate on a single relation(one operator)
 - **Binary** : Operate on two relations(two operator)

Select Operation(σ)

- Symbol: σ (sigma)
- **Unary** operator
- Notation : $\sigma_p(r)$ where p is called section condition or predicate and r is a relation
- The selection condition acts as a filter
 - Keeps only those tuples that satisfy the qualifying condition
 - Tuples satisfying the condition are selected whereas the other tuples are discarded (filtered out)
- Selects a subset of rows or tuples from relation according to a given selection condition or predicate.
- It is used as an expression *to choose tuples which meet the selection condition.*



Examples:

- Select the EMPLOYEE tuples whose department number is 4:
 $\sigma_{\text{DNO} = 4}(\text{EMPLOYEE})$
- Select the employee tuples whose salary is greater than \$30,000:
 $\sigma_{\text{SALARY} > 30,000}(\text{EMPLOYEE})$

Example

emp

<u>eno</u>	ename	sal	dno
1	Jones	1000	5
2	Smith	2500	1
3	Blake	3700	3
4	Xyp	2900	3
5	Pst	3785	5

Write the relational algebra expression to find the details of the emp (eno, ename, sal, dno) whose salary is 2500.

Ans: $\sigma_{(sal = 2500)}(emp)$

<u>eno</u>	ename	sal	dno
2	Smith	2500	1

External Query: select * from emp where sal = 2500

Example

emp

<u>eno</u>	ename	sal	dno
1	Jones	1000	5
2	Smith	2500	1
3	Blake	3700	3
4	Xyp	2900	3
5	Pst	3785	5

We can combine more than one condition using AND(^), OR(v), NOT(¬)

Write the relational algebra expression to find the details of the emp (eno, ename, sal, dno) whose sal > 3000 and dno = 5

Ans: $\sigma_{(sal > 3000 \wedge dno = 5)}(emp)$

<u>eno</u>	ename	sal	dno
5	Pst	3785	5

Example

r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	2	3
β	β	23	10

What is the output of the following relational algebra expression?

$\sigma_{(A = B \wedge D > 5)}(r)$

A	B	C	D
α	α	1	7
β	β	23	10

Example

<u>sid</u>	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

S2

sid	sname	rating	age
28	yuppy	9	35.0
58	rusty	10	35.0

$\sigma_{rating > 8}(S2)$

Example

- $\sigma_{DNO=5}(\sigma_{salary>25000}(\text{EMPLOYEE})) == \sigma_{salary>25000}(\sigma_{Dno=5}(\text{EMPLOYEE})) == \sigma_{DNO=5 \text{ AND } salary>25000}(\text{EMPLOYEE})$

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Projection Operation(π)

- PROJECT Operation is denoted by π (pi)
- **Unary** operator
- This operation keeps certain *columns* (attributes) from a relation and discards the other columns.
 - PROJECT creates a **vertical partitioning**
 - The list of **specified columns (attributes)** is kept in each tuple
 - The other attributes in each tuple are discarded
- The general form of the *project* operation is:
 - π (pi) is the symbol used to represent the *project* operation
 - $A_1, A_2, \dots, A_k$ are the desired list of attributes from relation R.
- The result is defined as relation of k columns obtained by eliminating **duplicate rows** from the result and **erasing the columns that are not listed**.
- Example: To list each employee(SSN, FNAME, LNAME, SALARY, ADDRESS, DNO) first and last name and salary, the following is used:

$\pi_{LNAME, FNAME, SALARY}(EMPLOYEE)$

Example

$\pi_{fields}(Input)$

Allows us to extract **columns** from a **relation**

Example:

<u>sid</u>	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

$\pi_{age}(S2)$

age
35.0
55.5

S2

Example

r

A	B	C
α	10	1
α	20	1
β	30	1
β	40	1

What is the output of the following relational algebra expression?

$\pi_{(A, C)}(r)$

A	C
α	1
β	1

Example

emp

<u>eno</u>	ename	sal	dno
1	Jones	1000	5
2	Smith	2500	1
3	Blake	3700	3
4	Xyp	2900	3
5	Pst	3785	5

Write the relational algebra expression to find employee name whose salary is greater than 2500.

Ans: $\pi_{\text{ename}} (\sigma_{\text{sal} > 2500}(\text{emp}))$

ename
Blake
Xyp
Pst

External Query: select ename from emp where sal > 2500

Example

emp

<u>eno</u>	ename	sal	dno
1	Jones	1000	5
2	Smith	2500	1
3	Blake	3700	3
4	Xyp	2900	3
5	Pst	3785	5

Write the relational algebra expression to find employee no and salary whose salary is greater than 3000 and dno = 3.

Ans: $\pi_{\text{ename, sal}} (\sigma_{(\text{sal} > 3000 \wedge \text{dno}=3)}(\text{emp}))$

ename	sal
Blake	3700

Project Operation Properties

- The number of tuples in the result of projection $\pi_{\langle \text{list} \rangle}(R)$ is always **less or equal** to the number of tuples in R
 - If the list of attributes includes a **key of R**, then the number of tuples in the result of PROJECT is **equal** to the number of tuples in R
- PROJECT is *not* **commutative**

Combination of Selection and Projection Operator

S

SNO	SNAME	AGE	STATE
S1	MIKE	21	IL
S2	STEVE	20	LA
S3	MARY	18	CA
S4	MING	19	NY
S5	OLGA	21	NY

$\pi_{SNO}(\sigma_{STATE='NY'}(S))$

O/P :

SNO
S4
S5

BASIC RELETIONAL ALGEBRA OPERATION FROM SET THEORY

NAME	SYMBOL
UNION	$\cup$
INTERSECTION	$\cap$
SET DIFFERENCE	$-$
CARTESIAN PRODUCT	$\times$

UNION, INTERSECTION, SET DIFFERENCE

- All of these operations take two input relations, which must be union-compatible:
 - Same number of fields.
 - Corresponding' fields have the same type.
- Two relation $R1(A1, A2, \dots, An)$ and $R2(B1, B2, \dots, Bn)$ are said to be union or type compatible, if they satisfy following two conditions:
 - Both the relations , i.e. $R1$ and $R2$ should be of **same degree**, i.e. they should have **same number of attributes**.
 - Domain of the **similar attributes**, i.e. **domain of i^{th} attribute of $R1$ and domain of i^{th} attribute of $R2$, should be same.** (i.e. $dom(A_i)=dom(B_i)$ for $i=1, 2, \dots, n$).
- The resulting relation for $R1 \cup R2$ (also for $R1 \cap R2$, or $R1 - R2$) has the same attribute names as the *first* operand relation $R1$ (by convention)

Union

- If R and S are two union compatible relation then the resultant relation $P = R \cup S$ is a relation includes all tuples that are **either in R or in S or in both R and S .**
Duplicate tuples are eliminated.
- It is defined as $R \cup S = \{t \mid t \in R \text{ or } t \in S\}$

R

A	B
α	1
α	2
β	2

S

A	B
α	2
β	3

$R \cup S$

A	B
α	1
α	2
β	2
β	3

Union

Account

Account_no	Name
101	Jones
103	Smith
104	Leon
107	Evan
110	Drew
112	Smita

Loan

Loan_no	Name
103	Smith
104	Leon
106	Byran
110	Drew

o/p

Name
Jones
Smith
Leon
Evan
Drew
Smita
Byran

Write the relational algebra expression to find name of those customer who have account or loan or both.

$$\Pi_{\text{Name}} (\text{Account}) \cup \Pi_{\text{Name}} (\text{Loan})$$

Union

S1

<u>sid</u>	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

S2

Union Compatible

<u>sid</u>	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0
44	guppy	5	35.0
28	yuppy	9	35.0

Compute: S1 U S2

S1(sid,sname,rating,age)

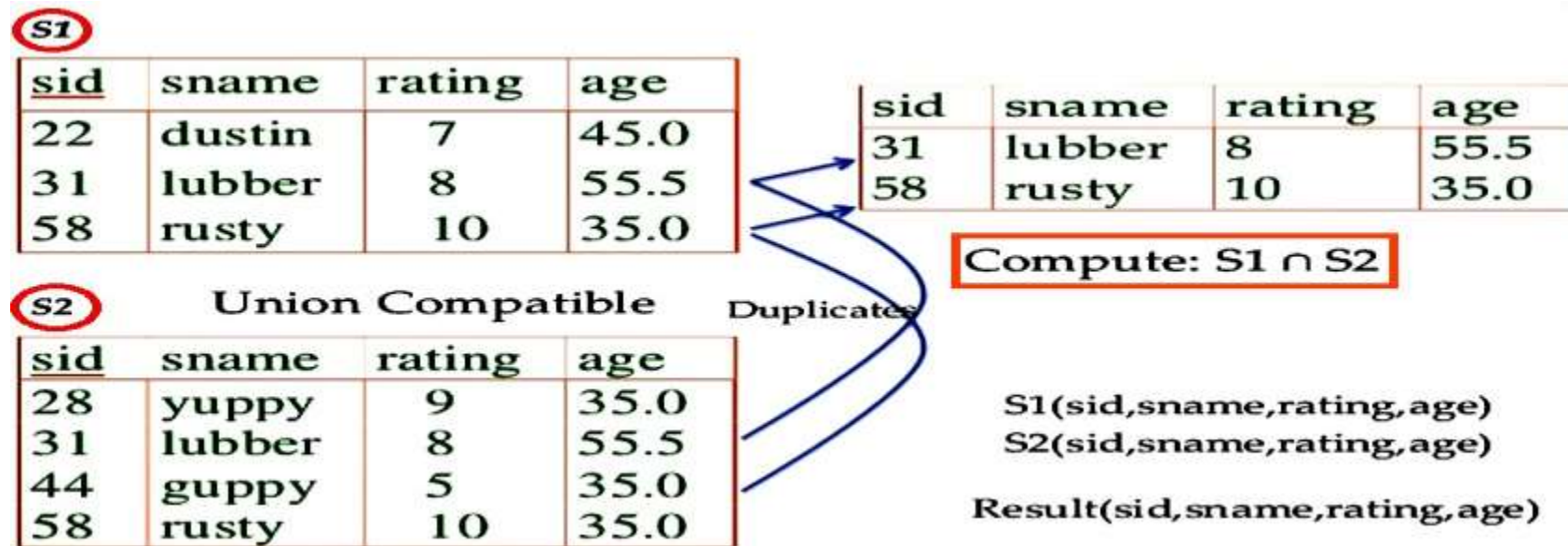
S2(sid,sname,rating,age)

Result(sid,sname,rating,age)

Duplicates

Intersection

- The result of this operation, denoted by $R \cap S$, is a relation that includes all tuples that are in both R and S .
- It is defined as $R \cap S = \{ t \mid t \in R \text{ and } t \in S \}$



Intersection

R

A	B
α	1
α	2
β	2

S

A	B
α	2
β	3

$R \cap S$

A	B
α	2

Intersection

Account

Account_no	Name
101	Jones
103	Smith
104	Leon
107	Evan
110	Drew
112	Smita

Loan

Loan_no	Name
103	Smith
104	Leon
106	Byran
110	Drew

o/p

Name
Smith
Leon
Drew

Write the relational algebra expression to find name of those customer who have both an account and loan.

$$\Pi_{\text{Name}}(\text{Account}) \cap \Pi_{\text{Name}}(\text{Loan})$$

Difference

- The result of this operation, denoted by $R - S$, is a relation that includes all tuples that are in R but not in S .
- It is defined as $R - S = \{ t \mid t \in R \text{ and } t \notin S \}$



Difference

R

A	B
α	1
α	2
β	2

S

A	B
α	2
β	3

R - S

A	B
α	1
β	2

Difference

Account

Account_no	Name
101	Jones
103	Smith
104	Leon
107	Evan
110	Drew
112	Smita

Loan

Loan_no	Name
103	Smith
104	Leon
106	Byran
110	Drew

o/p

Name
Jones
Evan
Smita

Write the relational algebra expression to find name of those customer who have but not taken a loan.

$$\Pi_{\text{Name}} (\text{Account}) - \Pi_{\text{Name}} (\text{Loan})$$

Properties of UNION, INTERSECTION and SET DIFFERENCE

- Notice that both union and intersection are **commutative** operations; that is
 - $R \cup S = S \cup R$, and $R \cap S = S \cap R$
- Both union and intersection can be treated as n-ary operations applicable to any number of relations as both are **associative operations**; that is
 - $R \cup (S \cup T) = (R \cup S) \cup T$
 - $(R \cap S) \cap T = R \cap (S \cap T)$
- The minus operation is not **commutative**; that is, in general
 - $R - S \neq S - R$

Cartesian Product or Cross Product

- Denoted by $\times$
- Let two relational schema $r(A_1, A_2, \dots, A_n)$ and $s(B_1, B_2, \dots, B_m)$
- Notation $r \times s$
- Defined as: $r \times s = \{t \mid t \in r \text{ and } t \in s\}$
 - Assumption is that the attributes of r and s are disjoint ($r \cap s = \Phi$)
- Each row of r combined with each row of s
- Result is a relation Q with degree $n + m$ attributes:
 - $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$, in that order.
- Hence, if r has n_r **tuples** (denoted as $|r| = n_r$), and s has n_s **tuples**, then $r \times s$ will have $n_r * n_s$ tuples or $|r \times s| = n_r * n_s$
- Also known as cross join
- It is not a meaningful operation unless it is associated with some condition.

Cartesian Product or Cross Product

Pair each tuple t_1 of R_1 with each tuple t_2 of S_1 .

<u>sid</u>	<u>bid</u>	<u>day</u>
22	101	10/10/96
58	103	11/12/96

$R_1(\text{sid}, \text{bid}, \text{day})$

<u>sid</u>	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

$S_1(\text{sid}, \text{sname}, \text{rating}, \text{age})$

$R_1 \times S_1$

	(sid)	sname	rating	age	(sid)	bid	day
1	22	dustin	7	45.0	22	101	10/10/96
2	22	dustin	7	45.0	58	103	11/12/96
3	31	lubber	8	55.5	22	101	10/10/96
4	31	lubber	8	55.5	58	103	11/12/96
5	58	rusty	10	35.0	22	101	10/10/96
6	58	rusty	10	35.0	58	103	11/12/96

Cartesian Product or Cross Product

r x s:

α	1
β	2

$$\sigma_{A=C}(r \times s)$$

α	10	<i>a</i>
β	10	<i>a</i>
β	20	<i>b</i>
γ	10	<i>b</i>

A	B	C	D	E
α	1	α	10	a
β	2	β	10	a
β	2	β	20	b

Cartesian Product or Cross Product

r x s:

α	1
β	2

$$\sigma_{A=C}(r \times s)$$

α	10	<i>a</i>
β	10	<i>a</i>
β	20	<i>b</i>
γ	10	<i>b</i>

A	B	C	D	E
α	1	α	10	a
β	2	β	10	a
β	2	β	20	b

Cartesian Product or Cross Product

stud

ID	NAME
1	XYZ
2	PQR

result

ID1	CGPA
1	9.2
2	8.5

Stud X result

ID	NAME	ID1	CGPA
1	XYZ	1	9.2
1	XYZ	2	8.5
2	PQR	1	9.2
2	PQR	2	8.5

Not Valid

Write the relational algebra expression to find student ID, name and CGPA.

Ans: $\pi_{\text{NAME, ID, CGPA}} (\sigma_{\text{ID} = \text{ID1}} (\text{stud} \times \text{result}))$

Cartesian product is meaningless unless it is associated with select condition.

ID	NAME	CGPA
1	XYZ	9.2
2	PQR	8.5

stud

ID	NAME
1	XYZ
2	PQR

result

ID1	CGPA
1	9.2
2	8.5

- Find the name of those student whose CGPA is >9.0

The relational algebra expression $\pi_{NAME} (\sigma_{cgpa > 9.0} (stud \times result))$ produces spurious tuples leading to incorrect result.

ID	NAME	ID1	CGPA
1	XYZ	1	9.2
1	XYZ	2	8.5
2	PQR	1	9.2
2	PQR	2	8.5

stud $\times$ result

ID	NAME	ID1	CGPA
1	XYZ	1	9.2
2	PQR	1	9.2

$\sigma_{cgpa > 9.0} (stud \times result)$

NAME
XYZ
PQR

$\pi_{NAME} (\sigma_{cgpa > 9.0} (stud \times result))$

stud

ID	NAME
1	XYZ
2	PQR

result

ID1	CGPA
1	9.2
2	8.5

- The correct relational algebra expression to find the name of those student whose CGPA is >9.0 is

$\pi_{NAME} (\sigma_{ID=ID1} (\sigma_{cgpa > 9.0} (stud \times result))).$

- Consider the following relational schema

Emp(eno,ename,dno,sal)

Dept(deptno,dname,dloc)

Que-Write the relational algebra expression to find the name all the employees who works in MCA department.

1 Emp × Dept

2 $\sigma_{\text{dname} = \text{'MCA'}} (\text{Emp} \times \text{Dept})$

3 $\sigma_{\text{dno} = \text{deptno}} (\sigma_{\text{dname} = \text{'MCA'}} (\text{Emp} \times \text{Dept}))$

4 $\pi_{\text{ename}} (\sigma_{\text{dno} = \text{deptno}} (\sigma_{\text{dname} = \text{'MCA'}} (\text{Emp} \times \text{Dept})))$

emp

<u>eno</u>	ename	sal	dno
1	Smith	5000	10
2	Jones	6000	20
3	Blake	7000	10
4	Peter	8000	30

dept

<u>deptno</u>	dname	dloc
10	MCA	bbsr
20	cs	ctc
30	IT	rkl

Join

- denoted by $\bowtie$
- Notation: $R \bowtie_{\langle \text{join condition} \rangle} S$.
- $R \bowtie_p S \equiv \sigma_p (R \times S)$

Join = Cartesian Product + Select

- The sequence of **CARTESIAN PRODECT** followed by **SELECT** is used quite commonly to identify and select related tuples from two relations
- The join condition must have an attribute of R and S.
- Only combination of tuples satisfying the join condition only appear in the result whereas in the cartesian product all combination of tuples are included in the results.
- If degree of R = n and degree of S = m then degree of $R \bowtie_p S = m+n$

Types of Join

- **Theta join**
- **Equi Join**
- **Natural Join**
- **Outer Join**
 - Left Outer Join
 - Right Outer Join
 - Full Outer Join

Theta Join Operation

The **Theta join** operation is an **extension** to the cross-join operation that allows us to combine a **selection** and a **cartesian product** into a **single operation**.

It defines a relation that contains tuples satisfying the **predicate** from the cartesian product of R and S.

- *Theta* can be any general Boolean expression on the attributes of R and S:

- $R.A_i < S.B_j$ AND $(R.A_k = S.B_l$ OR $R.A_p < S.B_q)$

$$r \bowtie_{\theta} s = \sigma_{\theta} (r \times s)$$

where, θ may be any of the comparison operator $<, \leq, >, \geq, =, \neq$

Equi join is special type of θ join where join condition uses only $=$ as the comparison operator.

Car

<u>Car_model</u>	Car_price
Car A	20,000
Car B	30,000
Car C	50,000

Boat

<u>Boat_model</u>	Boat_price
Boat 1	10,000
Boat 2	40,000
Boat 3	60,000

Car  Car_price > Boat_price Boat)

<u>Car_m</u> odel	Car_pr ice	Boat_ model	Boat_ price
Car A	20,000	Boat 1	10,000
Car B	30,000	Boat 1	10,000
Car C	50,000	Boat 1	10,000
Car C	50,000	Boat 2	40,000

Equijoin

Uses an **equality operator (=)** to join tables

When the join condition c contains only comparison operator $=$ then such type of join is called as EQUIJOIN.

Ex.

$$R \bowtie_c S = \sigma_c(R \times S)$$

Only $=$ operator can be used in join condition c .

Equi join is also a theta join.

1. Perform the cross product $S1 \times R1$
2. The perform the selection $R1.sid = S1.sid$

R1

<u>sid</u>	<u>bid</u>	<u>day</u>
2 2	1 0 1	1 0 / 1 0 / 9 6
5 8	1 0 3	1 1 / 1 2 / 9 6

S1

<u>sid</u>	sname	rating	age
2 2	dustin	7	4 5 . 0
3 1	lubber	8	5 5 . 5
5 8	rusty	1 0	3 5 . 0

$R1 \bowtie_{R1.sid = s1.sid} S1$

sid	Sname	Rating	age	sid	bid	age
22	dustin	7	45.0	22	101	10/10/96
58	rusty	10	35	58	103	11/12/96

The result of equi join we have one or more pair of attributes that have identical values in every tuple.

- Consider the following relational schema

Emp(eno,ename,**dno**,sal)

Dept(**deptno**,dname,dloc)

Que-Write the relational algebra expression to find the name and salary of all the employees who belong to MCA department.

1- $\text{Emp} \bowtie_{\text{dno}=\text{deptno}} \text{Dept}$

2 $\sigma_{\text{dname}='MCA'} (\text{Emp} \bowtie_{\text{dno}=\text{deptno}} \text{Dept})$

3 $\pi_{\text{ename}} (\sigma_{\text{dname}='MCA'} (\text{Emp} \bowtie_{\text{dno}=\text{deptno}} \text{Dept}))$

$\text{Emp} \bowtie_{\text{dno}=\text{deptno}} \text{Dept}$

<u>eno</u>	ename	sal	dno	<u>deptno</u>	dname	dloc
1	Smith	5000	10	10	MCA	bbsr
2	Jones	6000	20	20	cs	ctc
3	Blake	7000	10	10	MCA	bbsr
4	Peter	8000	30	30	IT	rkl

emp

eno	ename	sal	dno
1	Smith	5000	10
2	Jones	6000	20
3	Blake	7000	10
4	Peter	8000	30

dept

deptno	dname	dloc
10	MCA	bbsr
20	cs	ctc
30	IT	rkl

Natural Join Operation *

- Another variation of JOIN called NATURAL JOIN
- denoted by * or $\bowtie$ was created to get rid of the second (superfluous) attribute of an EQUIJOIN condition.
 - because one of each pair of attributes with identical values is superfluous in equijoin
- Natural join is an equi join of two relation R and S over all common attributes.
- The standard definition of natural join requires that the **two join attributes**, or **each pair of corresponding** join attributes, *have the same name and domain* in both relations
 - **NATURAL JOIN** does not use any of the comparison operators.
 - There should be at least one common attribute between two relations.
 - One occurrence of each common attribute is eliminated from the result.
 - It performs selection forming equality on those attributes which appear in both relations and eliminates the duplicate attributes.

Natural Join

<u>sid</u>	<u>bid</u>	<u>day</u>
22	101	10/10/96
58	103	11/12/96

R1(sid,bid,day)

<u>sid</u>	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

S1(sid,sname,rating,age)

$S1 \bowtie R1$

sid	sname	rating	age	bid	day
22	dustin	7	45.0	101	10/10/96
58	rusty	10	35.0	103	11/12/96

emp ⋈_{emp.deptno = dept.deptno} dept

eno	ename	sal	deptno	deptno	dname	dloc
1	Smith	5000	10	10	MCA	bbsr
2	Jones	6000	20	20	cs	ctc
3	Blake	7000	10	10	MCA	bbsr
4	Peter	8000	30	30	IT	rkl

emp * dept

<u>eno</u>	ename	sal	<u>deptno</u>	dname	dloc
1	Smith	5000	10	MCA	bbsr
2	Jones	6000	20	cs	ctc
3	Blake	7000	10	MCA	bbsr
4	Peter	8000	30	IT	rkl

<u>eno</u>	ename	sal	deptno
1	Smith	5000	10
2	Jones	6000	20
3	Blake	7000	10
4	Peter	8000	30

dept

<u>deptno</u>	dname	dloc
10	MCA	bbsr
20	cs	ctc
30	IT	rkl

Example

branch (branch_name, branch_city, assets)

customer (customer_name, customer_street, customer_city)

account (account number, **branch name**, balance)

loan (loan number, **branch name**, amount)

depositor (**customer name**, **account number**)

borrower (**customer name**, **loan number**)

Q) Find all loans of amount over \$1200

Q) Find the loan number for each loan amount greater than \$1200

q) Find the name, loan no and loan amount of all customers who have a loan at the bank



q) Find names of all customers who have either a loan and deposit account at the bank.



branch (branch_name, branch_city, assets)
customer (customer_name, customer_street, customer_city)
account (account number, branch name, balance)
loan (loan number, branch name, amount)
depositor (customer name, account number)
borrower (customer name, loan number)

Q) Find names of all customers who have a loan at 'DUMDUM' branch.

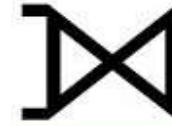
Q) Find names of all customers who have an account at 'Redwood' branch.

Outer Join

- An extension of the join operation that avoids loss of information.
- Computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join.

- In NATURAL JOIN and EQUIJOIN, tuples without a *matching* (or *related*) tuple are eliminated from the join result
 - Tuples with null in the join attributes are also eliminated. This amounts to loss of information.
- A set of operations, called OUTER joins, can be used when we want to keep all the tuples in R, or all those in S, or all those in both relations in the result of the join, regardless of whether or not they have matching tuples in the other relation.
- It uses NULL Values.

Left Outer Join



Matching Tuple in natural join + Extra tuple in the left relation.

Takes all the tuples in left relation that did not match with any tuple in the right relation.

The attributes of the relation for the unmatched tuples filled with NULL Values

t1		t2		t1 ⋈ _{LO} t2		
a	b	b	c	a	b	c
a1	b1	b1	c1	a1	b1	c1
a2	b2	b2	c2	a2	b2	c2
a3	b3	b4	c4	a3	b3	null

The left outer join operation keeps every tuple in the first or left relation R in R ⋈_{LO} S; if no matching tuple is found in R, then the attributes of S in the natural join result are filled or “padded” with null values.

Right Outer Join



Matching Tuple in natural join + Extra tuple in the right relation.

Takes all the tuples in right relation that did not match with any tuple in the right relation.

The attributes of the left relation for the unmatched tuple filled with NULL Values

t1

a	b
a1	b1
a2	b2
a3	b3

t2

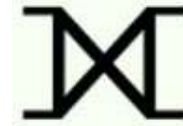
b	c
b1	c1
b2	c2
b4	c4

t1 ⋈_⌋ t2

a	b	c
a1	b1	c1
a2	b2	c2
<i>null</i>	b4	c4

The left outer join operation, keeps every tuple in the second or right relation S in the result of $R \bowtie S$. if no matching tuple is found in S, then the attributes of R in the natural join result are filled or “padded” with null values.

Full Outer Join



Matching Tuple in natural join + Extra tuple in both left and right relation.
 FULL OUTER JOIN = LEFT OUTER JOIN + RIGHT OUTER JOIN

t1

a	b
a1	b1
a2	b2
a3	b3

t2

b	c
b1	c1
b2	c2
b4	c4

t1 ⋈ t2

a	b	c
a1	b1	c1
a2	b2	c2
a3	b3	<i>null</i>
<i>null</i>	b4	c4

keeps all tuples in both the left and the right relations when no matching tuples are found, padding them with null values as needed.

Outer Join – Example

loan

<i>loan_number</i>	<i>branch_name</i>	<i>amount</i>
L-170	Downtown	3000
L-230	Redwood	4000
L-260	Perryridge	1700

Relation *borrower*

<i>customer_name</i>	<i>loan_number</i>
Jones	L-170
Smith	L-230
Hayes	L-155

loan * *borrower*

<i>loan_number</i>	<i>branch_name</i>	<i>amount</i>	<i>customer_name</i>
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith

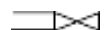
Outer Join – Example

loan

<i>loan_number</i>	<i>branch_name</i>	<i>amount</i>
L-170	Downtown	3000
L-230	Redwood	4000
L-260	Perryridge	1700

Relation *borrower*

<i>customer_name</i>	<i>loan_number</i>
Jones	L-170
Smith	L-230
Hayes	L-155

loan  *borrower*

<i>loan_number</i>	<i>branch_name</i>	<i>amount</i>	<i>customer_name</i>
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perryridge	1700	null

Outer Join – Example

loan

<i>loan_number</i>	<i>branch_name</i>	<i>amount</i>
L-170	Downtown	3000
L-230	Redwood	4000
L-260	Perryridge	1700

Relation *borrower*

<i>customer_name</i>	<i>loan_number</i>
Jones	L-170
Smith	L-230
Hayes	L-155

□ Right Outer Join

loan ⋈ *borrower*

<i>loan_number</i>	<i>branch_name</i>	<i>amount</i>	<i>customer_name</i>
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-155	<i>null</i>	<i>null</i>	Hayes

Outer Join – Example

loan

<i>loan_number</i>	<i>branch_name</i>	<i>amount</i>
L-170	Downtown	3000
L-230	Redwood	4000
L-260	Perryridge	1700

Relation *borrower*

<i>customer_name</i>	<i>loan_number</i>
Jones	L-170
Smith	L-230
Hayes	L-155

□ Full Outer Join

loan ⋈ *borrower*

<i>loan_number</i>	<i>branch_name</i>	<i>amount</i>	<i>customer_name</i>
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perryridge	1700	<i>null</i>
L-155	<i>null</i>	<i>null</i>	Hayes

RENAME OPERATION (ρ)

- The RENAME operator is denoted by ρ (rho)
- The rename operation allows to name the results of a relational algebra expression and also individual column.
- **Notation:** $\rho_x(E)$

-

RENAME OPERATION (ρ)

- In some cases, we may want to *rename* the attributes of a relation or the relation name or both
 - Useful when a query requires multiple operations
- Rename operation can be expressed by any of the following forms:
 - $\rho_S(B_1, B_2, \dots, B_n)(R)$ changes both:
 - the relation name to S , *and*
 - the column (attribute) names to $B_1, B_1, \dots, B_n$
 - $\rho_S(R)$ changes:
 - the *relation name* only to S
 - $\rho_{(B_1, B_2, \dots, B_n)}(R)$ changes:
 - the *column (attribute) names* only to $B_1, B_1, \dots, B_n$

Aggregate Functions and Operations

- **Aggregation function** takes a collection of values and returns a single value as a result.

avg: average value

min: minimum value

max: maximum value

sum: sum of values

count: number of values

- **Aggregate operation** in relational algebra

$$G_1, G_2, \dots, G_n \quad \mathcal{G} \quad F_1(A_1), F_2(A_2), \dots, F_n(A_n) (E)$$

E is any relational-algebra expression

- $G_1, G_2, \dots, G_n$ is a list of attributes on which to group (can be empty)
- Each F_i is an aggregate function
- Each A_i is an attribute name

- Note: Some books/articles use γ instead of $\mathcal{G}$ (C_Glligraphic $\mathcal{G}$)

Aggregate Operation – Example

- Relation r :

A	B	C
α	α	7
α	β	7
β	β	3
β	β	10

$G_{\text{sum}(c)}(r)$

sum(c)
27

Aggregate Operation – Example

- Find the average salary in each department

dept_name G **avg**(salary) (*instructor*)

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

<i>dept_name</i>	<i>avg_salary</i>
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

Aggregate Functions (Cont.)

- Result of aggregation does not have a name
 - Can use rename operation to give it a name
 - For convenience, we permit renaming as part of aggregate operation

dept_name $\mathcal{G}$ **avg**(salary) **as** *avg_sal* (*instructor*)

- Write the relational algebra expression to find the maximum salary of employees.

emp

<u>eno</u>	sal	ename	deptno
1	1000	xyz	d1
2	5000	pqr	d2
3	7000	abc	d1

$\sigma_{\max(\text{sal})}(\text{emp})$

$\max(\text{sal})$
7000

- Write the relational algebra expression to display the maximum salary of each department.

emp

<u>eno</u>	sal	ename	deptno
1	1000	xyz	d1
2	5000	pqr	d2
3	7000	abc	d1

deptno g_{max(sal)} (emp)

deptno	max(sal)
d1	7000
d2	5000

select max(sal) from emp group by deptno

Write RA expression to display deptno, avg salary and no of employees of each department

$\rho_{R(Dno, NO-of-Employees, Average-salary)} (Dno \text{ } g_{count(SSN),avg(salary)} (employee))$

R

Dno	No_of_employees	Average_sal
5	4	33250
4	3	31000
1	1	55000

$g_{count(SSN),avg(salary)} (employee)$

Count_ssn	Average_salary
8	35125

<u>SSN</u>	eno	salary	Dno
E1	SMITH	30000	5
E2	Wang	40000	5
E3	John	25000	4
E4	Jenifer	43000	4
E5	Rames h	38000	5
E6	Joyce	25000	5
E7	Ahmad	25000	4
E8	James	55000	1

- EMP(eid, ename, sal, city, dno)

Write relational algebraic expression to:

- a) Get the maximum, minimum and average salary of the employees.
- b) Find the total salary of employees for each department.
- c) Find the number of cities where the employees stay.

- **g** `max(Salary)` (**EMPLOYEE**) retrieves the maximum salary value from the EMPLOYEE relation
- **g** `min(Salary)` (**EMPLOYEE**) retrieves the minimum Salary value from the EMPLOYEE relation
- **g** `sum(Salary)` (**EMPLOYEE**) retrieves the sum of the Salary from the EMPLOYEE relation
- **g** `count(SSN),avg(Salary)` (**EMPLOYEE**) computes the count (number) of employees and their average salary

EMPLOYEE(SSN, Fname, Lname, salary, super_ssn, Dno)

DEPENDENT(ESSN, Dependent_name, Relationship)

- Display the names of all employees with two or more dependents.
 - $T1(Ssn, No_of_dependents) \leftarrow \text{Essn} \bowtie \text{COUNT}(Dependent_name)$
(DEPENDENT)
 - $T2 \leftarrow \sigma_{No_of_dependents > 2}(T1)$
 - $RESULT \leftarrow \pi_{Lname, Fname}(T2 * EMPLOYEE)$

Modification of Database

- The content of database may be modified using following operations:
 - **Deletion**
 - **Insertion**
 - **Updating**
- All these operations are expressed using the **assignment operator**.

Deletion

- A delete request is expressed similarly to a query.
- Instead of displaying tuples to the user, the selected tuples are removed from the database.
- Can delete the whole tuple; cannot delete values on only particular attributes
- A deletion is expressed in relational algebra by:

$$r \leftarrow r - E$$

where r is a relation and E is a relational algebra expression.

Deletion Examples

- Delete all account records of Perryridge branch.

***account* $\leftarrow$ *account* $- \sigma_{branch_name = \text{"Perryridge"}}(\textit{account})$**

- Delete all loan records with amount in the range of 0 to 50

***loan* $\leftarrow$ *loan* $- \sigma_{amount \geq 0 \text{ and } amount \leq 50}(\textit{loan})$**

Insertion

- To insert data into a relation, we either:
 - specify a tuple to be inserted
 - write a query whose result is a set of tuples to be inserted
- In relational algebra, an insertion is expressed by:

$$r \leftarrow r \cup E$$

where r is a relation and E is a relational algebra expression.

Insertion Examples

- Insert information in the database specifying that Smith has \$1200 in account A-973 at the Perryridge branch.

$account \leftarrow account \cup \{(\text{"A-973"}, \text{"Perryridge"}, 1200)\}$

$depositor \leftarrow depositor \cup \{(\text{"Smith"}, \text{"A-973"})\}$

Updating

- A mechanism to change a value in a tuple without changing *all* values in the tuple
- Use the generalized projection operator to do this task

$$r \leftarrow \Pi_{F_1, F_2, \dots, F_l}(r)$$

Update Examples

- Make interest payments by increasing all balances by 5 percent.

$account \leftarrow \Pi_{account_number, branch_name, balance * 1.05} (account)$

- Pay all accounts with balances over \$10,000 6 percent interest and pay all others 5 percent

$account \leftarrow \Pi_{account_number, branch_name, balance * 1.06} (\sigma_{BAL > 10000} (account)) \cup \Pi_{account_number, branch_name, balance * 1.05} (\sigma_{BAL \leq 10000} (account))$

Aggregate Operation – Example

- Relation r :

A	B	C
α	α	7
α	β	7
β	β	3
β	β	10

□ $g_{\text{sum}(c)}(r)$

sum(c)
27

If no grouping attributes are specified, the functions are applied to *all the tuples* in the relation, so the resulting relation has a *single tuple only*.

Delete

where r is a relation and E is a relational algebra expression.

- $r \leftarrow r - E$
- A delete is expressed similar to a query except that the result is removed from the database
- Cannot remove single attributes

Deletion Examples

account (account number, **branch name**, balance)

loan (loan number, **branch name**, amount)

- Delete all account records of Perryridge branch.

$account \leftarrow account - \sigma_{branch_name = "Perryridge"}(account)$

Delete all loan records with amount in the range of 0 to 50

$loan \leftarrow loan - \sigma_{amount \geq 0 \text{ and } amount \leq 50}(loan)$

Delete all employees record whose salary ≥ 5000 from emp(eno, ename, sal, dno) relation

$emp \leftarrow emp - \sigma_{sal \geq 5000}(emp)$

Insert

Syntax:

$$r \leftarrow r \cup E$$

where r is a relation and E is a relational algebra expression.

It is used to insert a new record into a relation.

Write the relational algebra expression to insert a **emp tuple(305,"rama",10000,10)** into emp(eno, ename, sal,dno).

$\text{emp} \leftarrow \text{emp} \cup \{305, \text{"rama"}, 10000, 10\}$

Insertion Examples

- Insert information in the database specifying that Smith has \$1200 in account A-973 at the Perryridge branch. Consider the following relational schema.

account (account number, branch name, balance)

depositor (customer name, account number)

$account \leftarrow account \cup \{("A-973", "Perryridge", 1200)\}$

$depositor \leftarrow depositor \cup \{("Smith", "A-973")\}$

UPDATE

A mechanism to change a value in a tuple without changing *all* values in the tuple

Use the generalized projection operator to do this task

$$r \leftarrow \Pi_{F_1, F_2, \dots, F_n}(r)$$

Where each F_i is either the i th attribute or updated value of i th attribute of r .

Write a relational algebra expression to increase the salary of each **employee**(eno,ename,sal,dno) by 30% .

employee $\leftarrow \Pi_{\text{eno}, \text{ename}, \text{sal} + 0.3 * \text{sal}, \text{dno}}$ (**employee**)

Update Examples

account (account_number, branch_name, balance)

- Make interest payments by increasing all balances by 5 percent.

$account \leftarrow \Pi_{account_number, branch_name, balance * 1.05} (account)$

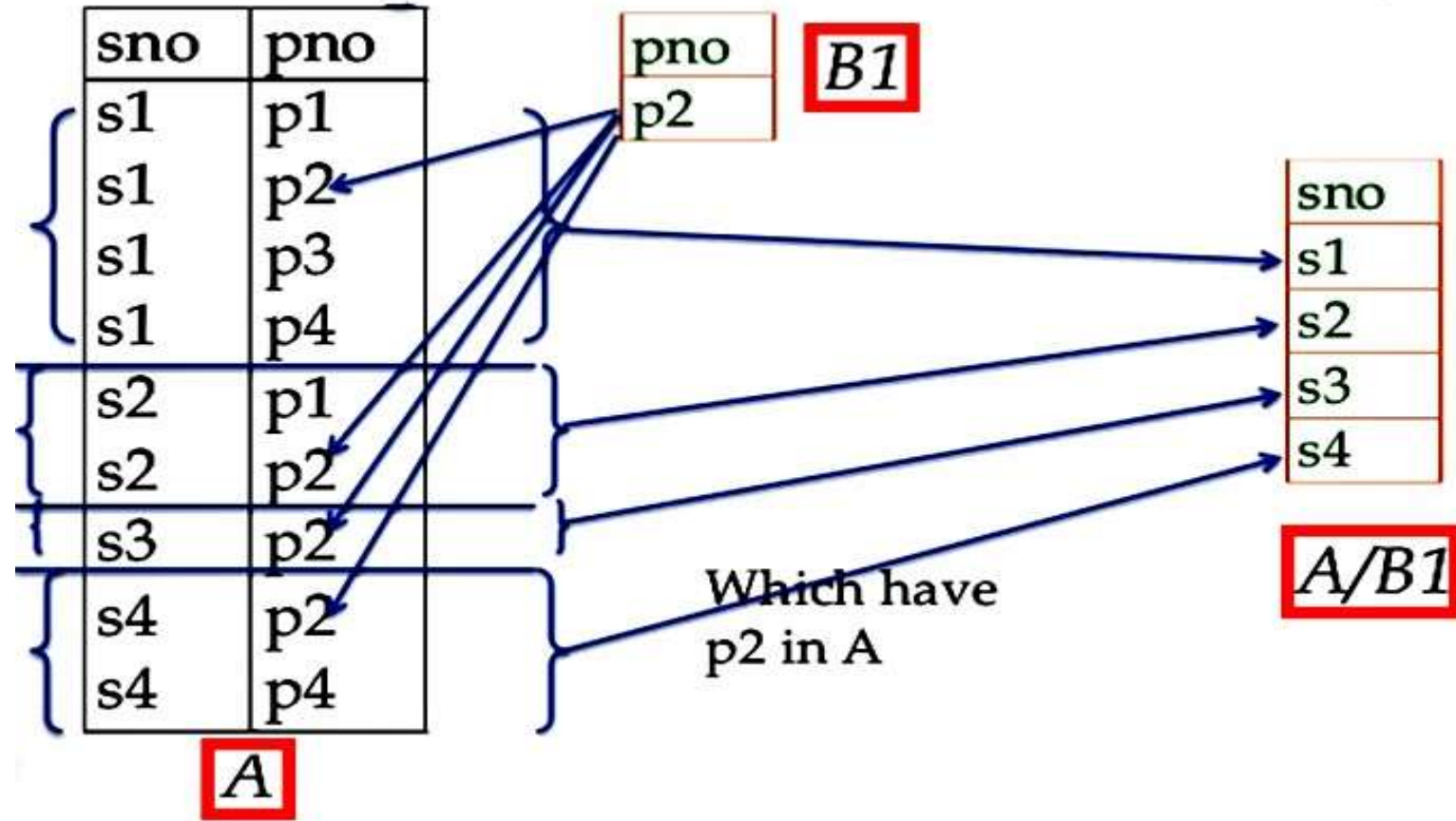
- Pay all accounts with balances over \$10,000 6 percent interest and pay all others 5 percent

$account \leftarrow \Pi_{account_number, branch_name, balance * 1.06} (\sigma_{BAL > 10000} (account)) \cup \Pi_{account_number, branch_name, balance * 1.05} (\sigma_{BAL \leq 10000} (account))$

Division Operation

- Division operator is denoted by 
- Notation: $r \div s$
- Suited for the queries that include the phrase **for all**
- It is a **binary operator**.
- $R(Z) \div S(X)$, where **X subset Z**. Let **Y = Z - X** (and hence $Z = X \cup Y$); that is, let **Y** be the set of attributes of R that are not attributes of S.
- The result of DIVISION is a relation **T(Y)** that includes a tuple t if tuples t_R appear in R with $t_R[Y] = t$, and with
 - $t_R[X] = t_s$ for every tuple t_s in S.
- For a tuple t to appear in the result T of the DIVISION, the values in t must appear in R in combination with every tuple in S.

Division Operation – Example



Division Operation – Example

Relations r, s :

R

A	B
a1	b1
a2	b1
a3	b1
a4	b1
a1	b2
a3	b2
a2	b3
a3	b3
a4	b3
a1	b4
a2	b4
a3	b4

S

A
a1
a2
a3

T

B
b1
b4

$$T \leftarrow R \div S.$$

A	B
α	1
α	2
α	3
β	1
γ	1
δ	1
δ	3
δ	4
ϵ	6
ϵ	1
β	2

r

B
1
2

s

$r \div s$:

A
α
β

Division Operation – Example

sno	pno
s1	p1
s1	p2
s1	p3
s1	p4
s2	p1
s2	p2
s3	p2
s4	p2
s4	p4

A

pno
p2

B1

sno
s1
s2
s3
s4

A/B1

pno
p2
p4

B2

sno
s1
s4

A/B2

pno
p1
p2
p4

B3

sno
s1

A/B3

Solving the query using **Assignment operator ($\leftarrow$)**

account (account_number, branch_name, balance)

depositor (customer_name, account_number)

branch (branch_name, branch_city, assets)

Query :

Find all customers who have an account at all branches located in Cuttack.

Solution :

$A \leftarrow \Pi_{\text{customer_name, branch_name}} (\text{depositor} * \text{Account})$

$B \leftarrow \Pi_{\text{branch_name}} (\sigma_{\text{branch_city} = \text{"Cuttack"}} \text{Branch})$

Result $\leftarrow A \div B$

Consider the following relational schema.

EMPLOYEE(SSN, Fname, Lname, salary, super_ssn, Dno)

WORKS_ON(ESSN, Pno, Hours)

*Retrieve the names of employees who work on **all the projects that ‘John Smith’ works on.***

SMITH $\leftarrow \sigma_{\text{Fname}='John' \text{ AND } \text{Lname}='Smith'}(\text{EMPLOYEE})$
SMITH_PNOS $\leftarrow \pi_{\text{Pno}}(\text{WORKS_ON} \bowtie_{\text{Essn}=\text{Ssn}} \text{SMITH})$
SSN_PNOS $\leftarrow \pi_{\text{Essn}, \text{Pno}}(\text{WORKS_ON})$
 $y \leftarrow \text{SSN_PNOS} \div \text{SMITH_PNOS}$
RESULT $\leftarrow \pi_{\text{Fname}, \text{Lname}}(y * \text{EMPLOYEE})$

- *Consider the following relational schema.*

EMPLOYEE(SSN, Fname, Lname, salary, super_ssn, Dno)

WORKS_ON(ESSN, Pno, Hours)

PROJECT(Pname, Pnumber, Plocation, Dnum)

- Find the names of employees who work on *all the projects controlled* by department number 5.

$DEPT5_PROJS \leftarrow \rho_{(Pno)}(\pi_{Pnumber}(\sigma_{Dnum=5}(PROJECT)))$

$EMP_PROJ \leftarrow \rho_{(ssn, Pno)}(\pi_{Essn, Pno}(WORKS_ON))$

$RESULT_EMP_SSNS \leftarrow EMP_PROJ \div DEPT5_PROJS$

$RESULT \leftarrow \pi_{Lname, Fname}(RESULT_EMP_SSNS * EMPLOYEE)$

- *Consider the following relational schema.*
- **EMPLOYEE(SSN, Fname, Lname, salary, super_ssn, Dno)**
DEPENDENT(ESSN, Dependent_name, Relationship)

Retrieve the names of employees who have no dependents.

- $ALL_EMPS \leftarrow \pi_{SSN}(EMPLOYEE)$
- $EMPS_WITH_DEPS(SSN) \leftarrow \pi_{ESSN}(DEPENDENT)$
- $EMPS_WITHOUT_DEPS \leftarrow (ALL_EMPS - EMPS_WITH_DEPS)$
- $RESULT \leftarrow \pi_{LNAME, FNAME}(EMPS_WITHOUT_DEPS * EMPLOYEE)$

Example

- *Consider the following relation schema:-*
- ***employee (person-name, street, city)***
- ***works (person-name, company-name, salary)***
- ***company (company-name, city)***
- ***manages (person-name, manager-name)***

Example

- Find *the names of all employees who work for First Bank Corporation*
- $\Pi_{person-name} (\sigma_{company-name = \text{"First Bank Corporation"}} (works))$
- Find the names and cities of residence of all employees who work for First Bank Corporation.
- $\Pi_{person-name, city} (employee * (\sigma_{company-name = \text{"First Bank Corporation"}} works))$
- $\equiv \Pi_{person-name, city} (employee \bowtie (\sigma_{company-name = \text{"First Bank Corporation"}} works))$
- $\equiv \Pi_{person-name, city} (\sigma_{company-name = \text{"First Bank Corporation"} \text{ AND } employee.person-name=works.person-name} employee \bowtie works)$
- $\equiv \Pi_{person-name, city} (\sigma_{company-name = \text{"First Bank Corporation"}} employee \bowtie_{employee.person-name=works.person-name} works)$

Example

- Find the names, street address, and cities of residence of all employees who work for First Bank Corporation and earn more than \$10,000 .
- $\Pi_{person-name, street, city}(\sigma_{(company-name = \text{"First Bank Corporation"} \wedge salary > 10000)}(works * employee))$
- Find the names of all employees in this database who live in the same city as the company for which they work.
- $\Pi_{person-name}(employee * works * company)$

Example

- Give all employees of First Bank Corporation a 10 percent salary raise.
- $works \leftarrow \Pi_{person-name, company-name, 1.1 * salary}(\sigma_{(company-name = \text{"First Bank Corporation"})}(works)) \cup (works - \sigma_{company-name = \text{"First Bank Corporation"}}(works))$

Example

- Consider the following relational schema

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

Example

- Query 1. Retrieve the name and address of all employees who work for the 'Research' department.
 - $\text{RESEARCH_DEPT} \leftarrow \sigma_{\text{Dname}='Research'}(\text{DEPARTMENT})$
 - $\text{RESEARCH_EMPS} \leftarrow (\text{RESEARCH_DEPT} \bowtie_{\text{Dnumber}=\text{Dno}} \text{EMPLOYEE})$
 - $\text{RESULT} \leftarrow \pi_{\text{Fname}, \text{Lname}, \text{Address}}(\text{RESEARCH_EMPS})$
- Query-2 For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.
 - $\text{STAFFORD_PROJS} \leftarrow \sigma_{\text{Plocation}='Stafford'}(\text{PROJECT})$
 - $\text{CONTR_DEPTS} \leftarrow (\text{STAFFORD_PROJS} \bowtie_{\text{Dnum}=\text{Dnumber}} \text{DEPARTMENT})$
 - $\text{PROJ_DEPT_MGRS} \leftarrow (\text{CONTR_DEPTS} \bowtie_{\text{Mgr_ssn}=\text{Ssn}} \text{EMPLOYEE})$
 - $\text{RESULT} \leftarrow \pi_{\text{Pnumber}, \text{Dnum}, \text{Lname}, \text{Address}, \text{Bdate}}(\text{PROJ_DEPT_MGRS})$

- Query 5. Make a list of project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

- $SMITHS(Essn) \leftarrow \pi_{Ssn} (\sigma_{Lname='Smith'}(EMPLOYEE))$
- $SMITH_WORKER_PROJS \leftarrow \pi_{Pno}(WORKS_ON * SMITHS)$
- $MGRS \leftarrow \pi_{Lname, Dnumber} (EMPLOYEE \bowtie_{Ssn=Mgr_ssn} DEPARTMENT)$
- $SMITH_MANAGED_DEPTS(Dnum) \leftarrow \pi_{Dnumber} (\sigma_{Lname='Smith'}(MGRS))$
- $SMITH_MGR_PROJS(Pno) \leftarrow \pi_{Pnumber}(SMITH_MANAGED_DEPTS * PROJECT)$
- $RESULT \leftarrow (SMITH_WORKER_PROJS \cup SMITH_MGR_PROJS)$